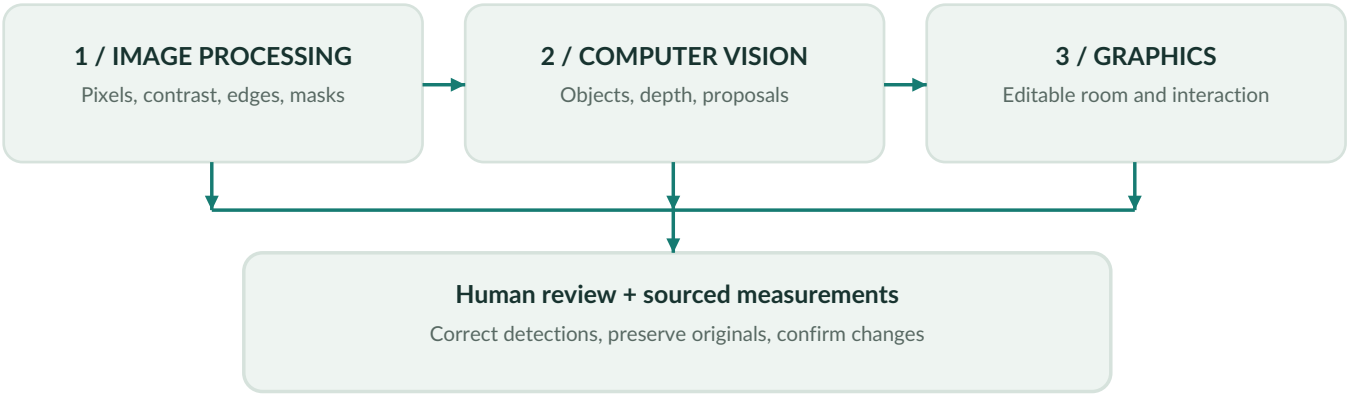
AuraSpace

End-to-end project guide for the viva

From a room photograph to a reviewed, editable furniture layout.

AuraSpace combines ordinary room photos, image inspection, learned furniture detection, optional depth estimation, furniture-removal previews, and an interactive room planner. The Android preview supports local planning with a processing backend; a Next.js web companion provides a separate browser workflow.



The three main sections

• **01 / Image processing** — what happens to pixels, why contrast and edges matter, how masks work, and how original pixels are protected.

• **02 / Computer vision** — what the models infer, why these models were selected, how calibration works, and what cannot be inferred reliably.

• **03 / Interactive computer graphics** — how the scene is represented, rendered and edited, plus the stack, Docker, Redis, persistence, API, evaluation and future improvements that support the experience.

What this handbook is based on

Repository source, the model manifest, recorded smoke outputs and implementation notes inspected on **25 October 2026**. Base Git commit: **f2a83e8**. Design rationales are technical interpretations of the implementation, rather than claims about the team's undocumented historical discussions.

Current status: installable Android preview, exercised in an emulator. Physical-phone camera/performance validation and independent real-room accuracy evaluation remain open. No invented accuracy, FPS or reconstruction results are used here.

Source trail: README.md; docs/implementation-audit.md; artifacts/verification.txt. External research references are listed on page 44.

How to study this guide

Read the problem and architecture first. Then follow the three subject sections. For each technique, learn five things: **input, transformation, output, reason for using it, and limitation**. That pattern answers most follow-up questions.

Pages	Coverage
3–4	Problem, features, full architecture and client differences
5–8	Image input, coordinates, contrast, edges, contours and morphology
9–13	Removal, real code, visual examples, alternatives and image-processing viva
14–18	Detection/segmentation, Mask R-CNN, browser SSD, model comparisons and LaMa
19–24	Depth, room geometry, calibration, identity, evaluation and computer-vision viva
25–30	Scene engine, procedural models, cameras, interaction, collision and circulation
31–36	Technology choices, Docker, Redis, durable jobs, storage, sync and security
37–42	Worked example, limitations, improvements, demo, viva answers and revision sheet
43–44	Repository navigation and external primary references

Three distinctions to keep clear

Pixels are not meters. A mask locates an object in a photo. It does not tell us that object's true width. Category defaults are editable priors; entered measurements have separate provenance.

A proposal is not an accepted scene change. Model jobs produce reviewable artifacts and observations. They do not automatically overwrite a newer layout or a user's measured values.

A rendered room is not an automatic scan. Current furniture is mostly procedural geometry. The room shell is editable. Multiple photos are supported, but they are not fused into a verified textured 3D reconstruction.

If you have only 30 minutes

Study pages 3–4, 15–17, 19–22, 25–29 and 32–35. Finish with the question pages 13, 24 and 41, and the revision sheet on page 42. Use the code map to open the implementation while rehearsing.

Source trail: README.md; packages/scene-domain/src/schema.ts; apps/worker/src/processor.ts.

The problem and the product

What problem does AuraSpace solve?

People often struggle to imagine whether existing furniture will fit a different room arrangement. Moving heavy pieces repeatedly takes effort; a photograph alone cannot be edited as a spatial plan. General CAD tools also demand manual modeling skills.

AuraSpace reduces this effort by turning visible furniture into **reviewable inventory**, keeping pieces in a tray, and letting users test layouts in inside, overview and plan views. Removal previews help imagine the room without selected movable furniture. Numeric dimensions and placement feedback support fit decisions.

Feature group	What the current implementation provides
Projects	Create, rename, search, examples, reopen, delete and portable native import/export
Capture and review	Camera/gallery photos, persistent originals, detections, masks, category correction, missed objects and identity merge/split
Room editing	Floor boundary, room height, openings, finishes, manual dimensions and four-corner floor review
Furniture	Tray, procedural items, custom boxes, dimensions, colors, locks, fixed fixtures, move/rotate, undo/redo and layout alternatives
Spatial feedback	Rotated overlap checks, boundary/ceiling warnings, conservative door clearance, walking and grid circulation
Outputs	Native ZIP with scene/media/hashes, dimensioned SVG and view sharing; web JSON/SVG/PNG
Backend	Private uploads, queued analysis/removal/processing jobs, retries, cancellation and explicit result review

Typical user journey

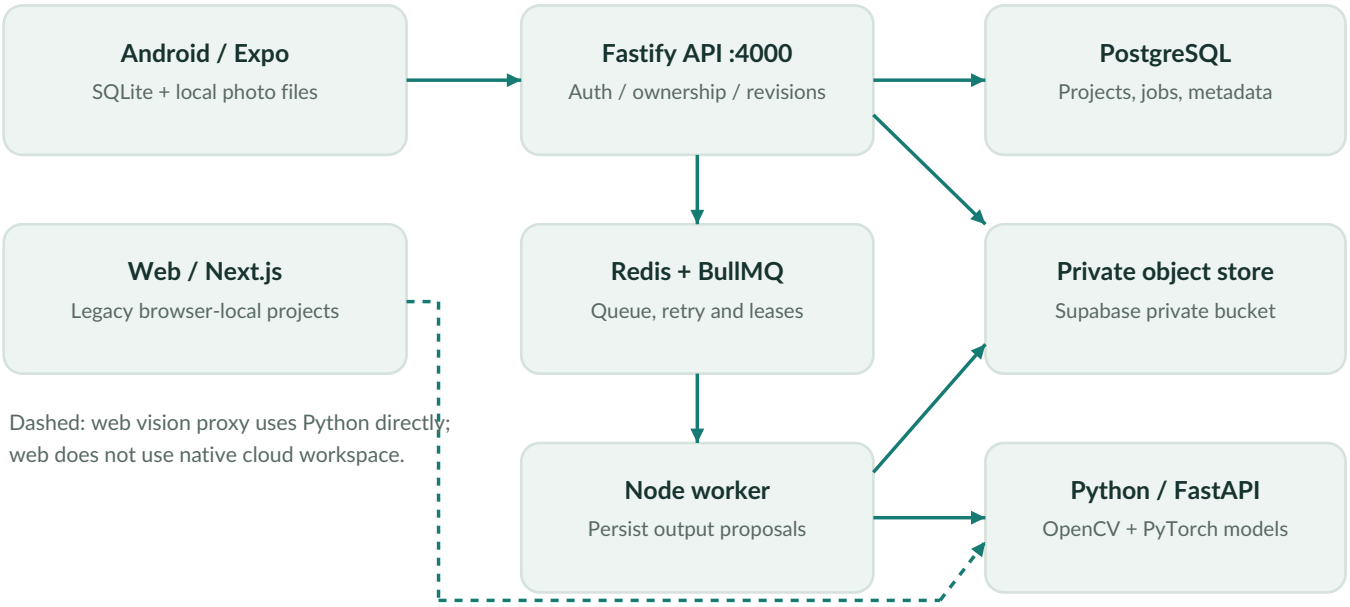
Create a room → import photos → analyze → correct observations and masks → link observations to physical items → enter known dimensions → keep movable pieces in the tray → edit the room → place and compare layouts → export.

Manual planning remains available without a configured analysis server. Detection is assistance: users can add a missed item or a custom box. Furniture categories in the editor exceed the classes recognized by the configured detector.

Viva opening: “We built an assisted room-planning system. Image processing prepares and edits photographs, computer vision proposes furniture and depth information, and graphics makes the reviewed scene spatially editable.”

Source trail: README.md; apps/mobile/app/room/[id].tsx; docs/capture-guide.md; docs/demo-script.md.

Architecture: how the parts connect



Read this diagram as two client paths

Android: the Expo app edits a shared scene document and saves locally using SQLite and app files. When configured, it talks to the Fastify API. The API checks ownership, accepts uploads and persists jobs. BullMQ dispatches those jobs through Redis to a Node worker. The worker calls the private FastAPI service, saves result images in object storage, and records proposals for review.

Web: the Next.js client keeps a legacy browser-local project format. It can call a server-side vision proxy or use browser COCO-SSD detection. It does not automatically share native SQLite data or behave as a second frontend to the native cloud workspace.

Why separate these responsibilities?

Rendering needs quick, local feedback; learned inference may take seconds and require large Python dependencies. The worker allows editing while analysis runs. A shared domain package centralizes native scene rules. Separate object storage keeps large binary images out of relational rows and Redis messages.

The local Compose setup uses private service networking. Database and Redis host ports bind to localhost; Python is reached internally. These development defaults are useful isolation, but hosted HTTPS, monitoring and real deployment validation are separate tasks.

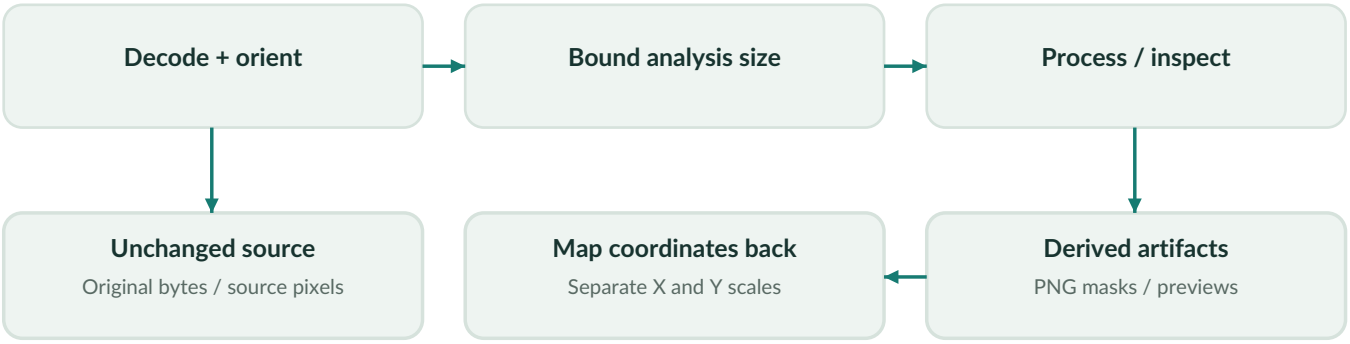
Source trail: docker-compose.yml; apps/api/src/app.ts; apps/worker/src/processor.ts; src/services/vision.web.ts; src/server/vision-proxy.ts.

01 / Image processing

What this subject contributes

Image processing changes or analyzes the numerical pixel representation of an image. In AuraSpace it supports **orientation, decoding, resizing, contrast inspection, edges, contours, histograms, binary masks, cutouts and controlled removal composition**.

Computer vision adds semantic interpretation: “these pixels belong to a sofa.” Graphics then creates an editable representation of the reviewed result. There is overlap between the subjects; segmentation and inpainting combine learned vision with image operations.



Input handling is part of correctness

The Python decoder checks the encoded payload length, decodes bytes, limits input to 20 megapixels, applies EXIF orientation with Pillow, converts to RGB and then to OpenCV's BGR order. Native capture accepts bounded sources and keeps smaller analysis copies for responsive review.

If a portrait photograph is rotated after its mask was computed, the mask no longer aligns with the furniture. Therefore observations use **oriented source-image pixels**, while capture metadata separately records analysis-copy scale and offsets.

What is preserved?

Original media is stored separately from generated derivatives. “Original bytes” and “oriented decoded pixels” describe different things: EXIF changes display orientation without requiring the original photo file to be overwritten. Removal composition preserves pixels outside the mask in the decoded image domain and exports a lossless PNG.

Memory example: a 4000 × 3000 BGR uint8 array needs about 36 MB before masks, float tensors or model activations. Limiting inference dimensions reduces intermediate memory and processing cost; it does not by itself bound every decoded allocation.

Source trail: services/vision/app/preprocessing.py; apps/mobile/src/features/capture/controller.ts; docs/capture-guide.md.

Luminance, contrast and CLAHE

Why convert color spaces?

OpenCV arrays use BGR; learned image models normally receive RGB. Grayscale removes chromatic channels to inspect intensity structure. The implemented grayscale conversion uses approximately **Y = 0.299R + 0.587G + 0.114B**. Green has the largest contribution to this luma approximation.

Simple contrast adjustment computes **p' = clip(mean + a(p − mean), 0, 255)**. The default factor is 1.35. It spreads values away from the image mean; clipping can discard highlight or shadow detail.

Why CLAHE?

Contrast-Limited Adaptive Histogram Equalization enhances contrast in local tiles and limits amplification. AuraSpace applies it to the **L channel of LAB**, retaining the a and b channels. This reduces direct modification of color channels, although quantization and conversion can still change the final RGB values. Defaults: clip limit **2.0**, tile grid **8 × 8**. [R1]

```
# Actual core of apply_clahe(), preprocessing.py
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
l_channel, a_channel, b_channel = cv2.split(lab)
clahe = cv2.createCLAHE(
    clipLimit=CLAHE_CLIP_LIMIT,
    tileGridSize=(CLAHE_TILE_SIZE, CLAHE_TILE_SIZE),
)
l_channel = clahe.apply(l_channel)
lab = cv2.merge([l_channel, a_channel, b_channel])
return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
```

Choice	Why it can help	Tradeoff / alternative
Global contrast	Simple, inexpensive intensity transform	Cannot adapt separately to dark and bright regions
Global equalization	Redistributes one image-wide intensity histogram	May handle uneven lighting poorly
CLAHE	Local contrast with amplification control	Can strengthen noise; requires parameter tuning

Implementation boundary: CLAHE is an inspection operation in `/process`. The current `/analyze` detector does not automatically run a CLAHE stage. An accuracy improvement from preprocessing would require an ablation study.

Source trail: services/vision/app/preprocessing.py: apply_clahe, enhance_contrast; app/routes.py: process_image, analyze_room. [R1]

Edges, contours and histograms

Edges locate intensity changes

Sobel approximates horizontal and vertical derivatives, Gx and Gy; their magnitude is sqrt($Gx^2 + Gy^2$). AuraSpace normalizes that magnitude to uint8 for display. It shows gradual and strong transitions, but it does not assign object meaning.

Canny combines smoothing, gradients, non-maximum suppression and two-threshold hysteresis. The repository first blurs with a 5 × 5 Gaussian kernel (sigma 1.4), then chooses thresholds from the blurred image median: **low = 0.66m**, **high = 1.33m**, bounded to 0–255. [R2]

One source comment calls this “Otsu,” but the executable code uses the median heuristic. In the viva, explain the code that actually runs. Canny's non-maximum suppression thins edges; detector NMS suppresses duplicate boxes. They are different operations.

Contours turn connected boundaries into geometry

The contour pipeline is Canny → 3 × 3 rectangular dilation → external contour tracing → minimum-area filtering → polygon simplification. The `/process` route defaults to a minimum area of **200 pixels²**; the helper's own default is 500. It keeps at most 50 contours.

`approxPolyDP` uses an epsilon equal to 1% of the contour perimeter in this helper. Simplifying outlines reduces storage and drawing work. A boundary around a patterned rug or wall frame is still just a contour, not a recognized furniture item.

Histograms summarize color distribution

The service computes 256-bin per-channel counts. They help explain exposure and tonal distribution, but discard spatial arrangement: two different rooms can have similar histograms. The histogram route returns the histogram metadata alongside a CLAHE image; that image is not a plotted histogram.

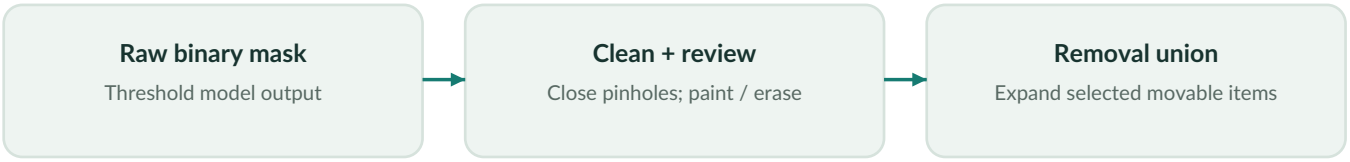
Technique	Output	Main limitation
Sobel / Canny	Gradient or thin edge map	Texture, shadows and reflections create misleading edges
Contours	Boundary points, area and bounding rectangle	A closed outline does not establish semantic identity
Histogram	Counts per intensity/channel	No knowledge of position, shape or room dimensions

Source trail: services/vision/app/preprocessing.py: detect_edges, extract_contours, compute_histogram; app/routes.py: process_image. [R2]

Masks and mathematical morphology

Why a mask instead of only a rectangle?

A bounding rectangle contains background around chair legs, arms and gaps. A binary instance mask selects furniture pixels more precisely. It supports transparent thumbnails and more controlled removal. A mask is H × W; **0 means preserve and 255 means foreground/remove**, depending on the operation.



White = remove / foreground. Black = preserve. Each instance keeps its own identity.

The four basic operations

- **Erosion:** shrinks white foreground; can remove tiny noise but also thin furniture legs.
- **Dilation:** expands white foreground; helps include object edges or selected shadows but may cover nearby features.
- **Opening = erosion then dilation:** removes small foreground specks.
- **Closing = dilation then erosion:** fills small holes and narrow gaps. [R3]

The detector thresholds its probability mask at **0.5**, maps it back to source resolution using nearest-neighbor interpolation, then applies one **3 × 3 closing**. Raw and cleaned masks are kept separately for inspection. The generic morphology helper exposes additional operations; those are not all applied to every detection.

Human mask review

Native review supports polygons, paint/erase strokes, boxes and expansion. Each selection is rasterized; strokes are applied in order, optional elliptical dilation expands it, and selected instance masks are combined with bitwise OR. Empty and full-image removal masks are rejected because surrounding context is required.

An erase stroke affects its own selection; union with another overlapping selection can restore white pixels. Fixed or ignored observations are excluded from native removal jobs. Mask revisions prevent processing an outdated selection.

Shape example: if a sofa mask misses its shadow, removal may leave a dark silhouette. Slight expansion can help. Excessive expansion can erase a floor seam or a nearby chair. Review beats assuming that more dilation is always better.

Source trail: services/vision/app/masks.py; app/inference.py; app/preprocessing.py: apply_morphology; apps/api/src/routes/jobs.ts. [R3]

Removing furniture from a photograph

The end-to-end removal operation

User selects reviewed movable observations → API validates ownership and mask revisions → worker loads the immutable scene snapshot → Python rasterizes selections → Big-LaMa is preferred if configured → labeled OpenCV fallback if necessary → composite only inside the mask → save PNG preview for explicit review.

Inpainting fills missing image content. It does not move an actual 3D object out of a reconstructed scene. Hidden tiles, skirting boards or wall textures are synthesized from visual context. Several physically different rooms could explain the same visible pixels.

Method	Reason to use it	Main tradeoff
Big-LaMa	Learned context for large missing regions and structural texture	Needs a substantial model; plausible output can be factually wrong
Telea	Fast classical neighborhood-based propagation	Large holes often smear or repeat nearby colors
Navier–Stokes	Classical continuation of image structure/isophotes	Can struggle with large missing semantic content
Directional fill	Custom inverse-distance sampling in eight directions	Slow pixel loops; not the automatic large-hole choice

What does “auto” mean here?

The preferred learned path is controlled by `prefer_lama` and local model availability. In the classical `remove_furniture(method='auto')` helper, a mask smaller than 5% of the image chooses Navier–Stokes; larger masks use Telea. Despite a historical comment mentioning directional fill for large holes, the executed fallback stays Telea.

The LaMa adapter caps its working side at 1024 pixels, pads dimensions to a multiple of eight, runs the TorchScript model and resizes the output. The final composition uses the source-size mask, so unselected decoded source pixels remain unchanged.

Why not just blur the sofa? Blur still shows its occupied region and does not reconstruct surrounding surfaces. Inpainting produces a more useful visualization, while the procedural planner remains the editable representation.

Source trail: apps/worker/src/processor.ts; services/vision/app/routes.py: inpaint_room; app/lama.py; app/inpainting.py.

Two details that prevent wrong results

1. Mapping analysis pixels back to source pixels

Suppose a source is 4000 × 3000 and detection runs at 1024 × 768. Then sx = 4000/1024 and sy = 3000/768. An analysis box corner at (256, 192) maps to source (1000, 750). Use actual resized dimensions: rounding can make sx and sy differ slightly.

```
# Actual excerpts from detect_furniture(), inference.py
sx, sy = source_w / analysis.shape[1], source_h / analysis.shape[0]
binary_mask = (masks[i, 0] > 0.5).astype(np.uint8) * 255
if binary_mask.shape != image.shape[:2]:
    binary_mask = cv2.resize(
        binary_mask, (source_w, source_h),
        interpolation=cv2.INTER_NEAREST)
# ... box coordinates are scaled separately:
x1, x2, y1, y2 = x1*sx, x2*sx, y1*sy, y2*sy
```

Nearest-neighbor interpolation retains discrete mask labels. Bilinear interpolation would introduce intermediate values, requiring a new threshold. Neither mapping yields physical meters; it only changes image coordinates.

2. Compositing protects unselected pixels

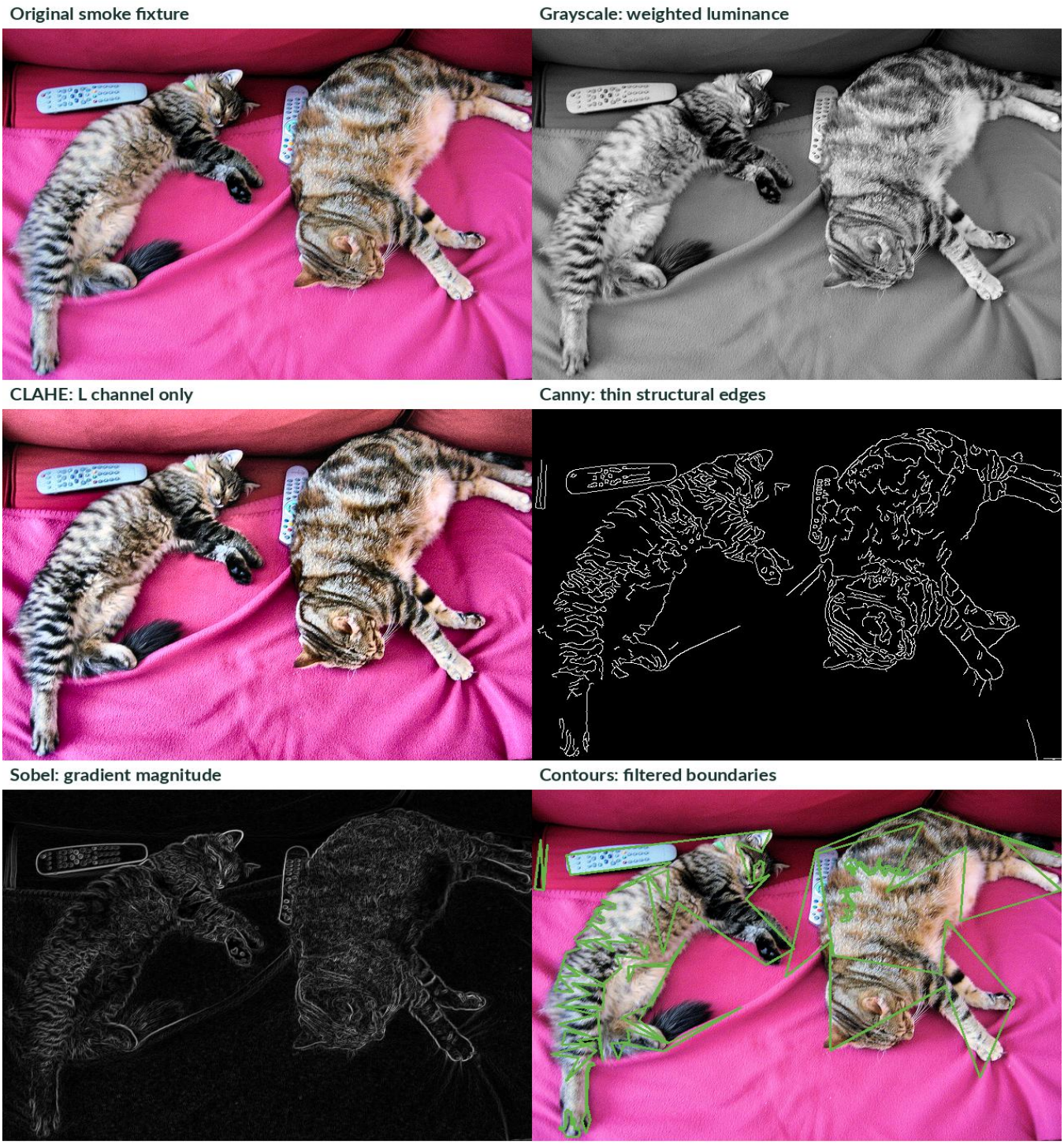
For a binary alpha mask M and candidate image G, **O = M·G + (1−M)·I**, where I is the decoded original. For M = 0, O equals I exactly.

```
# Actual core of composite_result(), inpainting.py
alpha = (mask > 0).astype(np.float32)[:, :, np.newaxis]
composited = (alpha * result.astype(np.float32)
              + (1.0 - alpha) * original.astype(np.float32))
return composited.astype(np.uint8)
```

The quoted alpha branch handles a two-dimensional mask. The full function also accepts a channel image. PNG output avoids JPEG recompression changing pixels outside the selection. Exact preservation refers to decoded pixels; it does not mean the output file's compressed bytes match the original JPEG.

Source trail: services/vision/app/inference.py: detect_furniture; app/inpainting.py: composite_result; app/routes.py: inpaint_room.

See the image-processing outputs



Actual outputs: generated for this handbook by calling the repository's grayscale, CLAHE, Canny, Sobel and contour functions. Input: public COCO val2017 image 39769, the same fixture named in the recorded model smoke report. These examples illustrate operations, not a room-quality benchmark.

The sofa's texture and other scene details generate many edges. This is why edge detection alone cannot decide “sofa,” identify the floor, or measure the room. Grayscale removes color; CLAHE changes local contrast; contour extraction converts some connected image boundaries into outlines.

Source trail: docs/viva/build_guide.py: make_assets; services/vision/app/preprocessing.py; reports/model-smoke.json. Image source link: page 44.

Image-processing choices and failure cases

Why OpenCV, NumPy and Pillow?

OpenCV supplies established image operations and image encoding. **NumPy** represents pixels and masks as arrays, enabling vectorized thresholding, statistics and composition. **Pillow** handles image decoding/EXIF orientation before metadata is discarded. Each library has a focused responsibility.

Writing every operation manually would increase debugging effort and make border behavior, color conversion and performance harder to verify. A pure browser implementation is useful for local convenience; the Python service offers the same inspection tools close to the learned model runtime. The web's lightweight pixel operations should not be described as identical implementations of all OpenCV stages.

Failure / tradeoff	How to explain and handle it
Dark or blurry image	Validation reports mean-luminance and Laplacian-variance warnings. Better capture can help; low texture can also lower variance.
Incorrect orientation	Apply EXIF before generating coordinates; verify stored dimensions during worker validation.
Noisy contrast enhancement	Reduce amplification; inspect the original. No claim that CLAHE always improves detection.
Mask joins nearby objects	Small cleanup and per-instance review reduce unwanted merging. Retain raw evidence.
Too-large removal	Expect weak or invented texture; refine selection or use manual room planning.
Large payload	Bound input pixels/bytes and inference copies; avoid treating a long base64 string as a small request.

How would we evaluate this section?

For masks, compare against independently annotated foreground using **IoU = intersection/union** and Dice overlap. For paired or synthetic inpainting, evaluate masked-region error/SSIM alongside outside-mask equality; evaluate real photographs with user ratings and correction effort because hidden ground truth is normally unavailable.

For preprocessing, run an **ablation**: same frozen photos and detector, with and without the operation. Report changes in precision/recall or mask quality, runtime and failure cases. Do not infer improvement from a more attractive contrast display.

Source trail: services/vision/app/preprocessing.py; app/masks.py; app/routes.py: validate_image; docs/evaluation.md.

Image-processing viva questions

Q1. Where exactly is image processing used?

During decode/orientation and resizing; in grayscale, contrast, CLAHE, edges, contours and histograms; in thresholded masks, morphology and cutouts; and in final removal composition. These operations manipulate pixels rather than assign a complete semantic room model.

Q2. Why is CLAHE applied to LAB lightness?

It focuses contrast adjustment on lightness while retaining the chromatic channels. Local tile equalization handles uneven intensity distributions; clipping limits amplification. It is an inspection option, not a mandatory detector preprocessing stage.

Q3. How is Canny different from Sobel?

Sobel estimates derivatives. Canny adds a staged process with smoothing, edge thinning and hysteresis to produce a cleaner edge map. Our thresholds use the image median; the implementation does not run Otsu thresholding there.

Q4. Why is closing useful for segmentation masks?

It fills small foreground holes. We use a small 3 × 3 closing once. Large kernels might merge separate objects or destroy thin details, so we retain raw masks and allow correction.

Q5. Does removing furniture recover the original wall?

It generates a plausible wall/floor appearance. A single photograph does not contain the hidden pixels, so the preview is visualization rather than physical evidence. The original remains available.

Q6. Why PNG after inpainting, and nearest-neighbor for masks?

PNG preserves the composited pixel values without JPEG loss. Nearest-neighbor resizing keeps binary masks discrete. Regular color images may use interpolating resizers because their pixels are continuous-valued samples rather than labels.

Q7. Is a contour the same as an object mask?

A contour is an outline extracted from connected image structure. A model instance mask assigns pixels to one detected object. We can convert masks to polygons, but contours from arbitrary edges do not automatically identify furniture.

Q8. What are the main formulas to remember?

Weighted grayscale; Sobel magnitude $\sqrt{G_x^2 + G_y^2}$; coordinate scaling $x_{source} = s_x \cdot x_{analysis}$; binary composition $O = M \cdot G + (1 - M) \cdot I$; and IoU = intersection/union.

Source trail: Image-processing implementation: services/vision/app/{preprocessing,masks,inpainting,inference,routes}.py.

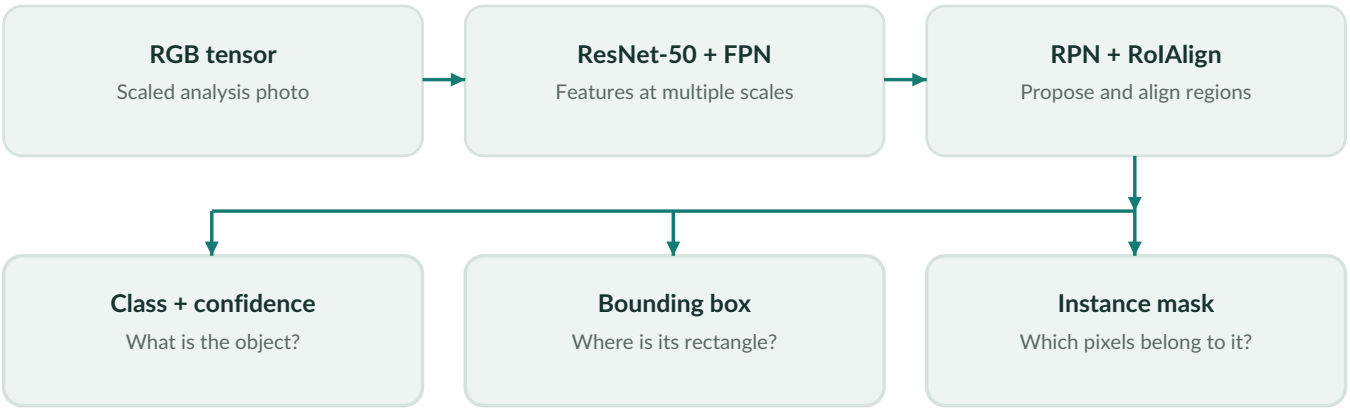
02 / Computer vision

What does the computer understand?

Computer vision estimates meaningful scene information from image evidence. AuraSpace uses learned models to propose **furniture instances**, optional **per-pixel depth**, and an **inpainted appearance**. Classical geometry helpers add line and vanishing-point evidence. Users review these outputs before treating them as useful project information.

Task	Question answered	AuraSpace role
Classification	What class is present?	Furniture category within each detected region
Object detection	What and where?	Label, confidence and bounding rectangle
Semantic segmentation	Which class at each pixel?	Conceptual comparison; not the primary configured output
Instance segmentation	Which pixels belong to each separate object?	Mask R-CNN keeps two chairs as distinct instances
Depth estimation	How far is each visible surface?	Optional learned metric indoor depth; needs validation
Inpainting	What could the missing appearance look like?	Big-LaMa furniture-removal preview

Main detector architecture



Mask R-CNN adds a mask-prediction branch to a region-based detector. ResNet-50 extracts features; the Feature Pyramid Network combines multiple spatial scales; the Region Proposal Network proposes regions; RoIAlign samples aligned region features. Classification/box and mask heads provide complementary outputs. [R4]

The research architecture explains the model; the project reuses a pretrained implementation. It does not implement Mask R-CNN training, its loss functions or the backbone from scratch. The contribution is the complete capture, review, identity, planning and durable-processing workflow around inference.

Source trail: services/vision/app/inference.py; services/vision/model-manifest.json. Architecture background: Mask R-CNN paper [R4].

The exact detector we use

Server model and configuration

The pinned server detector is **torchvision Mask R-CNN ResNet-50 FPN v2, COCO_V1**. The checkpoint filename is `maskrcnn_resnet50_fpn_v2_coco-73cbd019.pth`. The manifest records its complete SHA-256; configured loading verifies this digest before loading the state dictionary.

Parameter	Current default / meaning
Furniture class IDs	62 chair; 63 couch; 64 potted plant; 65 bed; 67 dining table
Detection score	At least 0.5; controls which predictions are returned
Additional box NMS	Class-aware, IoU threshold 0.4
Detection cap	25 per image
Instance mask cutoff	Probability above 0.5 becomes foreground
Working image	Longer side limited to 1024 in detect_furniture
Runtime	eval mode, no gradients; configured device, CPU in Compose

COCO has 80 object categories; torchvision's label indexing includes gaps and background in a larger lookup list. Therefore furniture IDs above are not indices 0–4 and should not be renumbered casually.

Why this model fits the requirement

The workflow needs both boxes and masks. A box identifies and labels a candidate; its instance mask enables cutouts, editable removal selections and foreground statistics. Existing COCO furniture categories make pretrained inference useful without collecting a training dataset first.

The costs are model size, inference latency and limited category coverage. Cabinets, custom furniture and rugs may be available in the editor but are not among the configured five server classes. Occlusion, unusual viewpoints and local furniture styles can still fail.

Did we train or fine-tune it?

No project-specific training is implemented. We reuse pretrained weights and filter outputs. Calling this reuse of learned representations is reasonable; claiming that the team trained or fine-tuned a custom detector would be inaccurate. Model confidence is a ranking score, not a guaranteed calibrated probability of correctness.

The loader also has an upstream-default path when no local checkpoint is configured. The documented deployment supplies verified local paths; that is the reproducible path explained here.

Source trail: services/vision/app/config.py; app/inference.py; services/vision/model-manifest.json; docker-compose.yml.

The inference pipeline, step by step

From decoded photograph to observations

1. Preserve oriented source dimensions. Resize only the inference copy if needed.

2. Convert BGR to RGB, then to a float tensor with values in the usual 0–1 input range.

3. Run the torchvision model with gradients disabled. Internal model transforms still apply.

4. Filter outputs by the five configured furniture IDs and the 0.5 score cutoff.

5. Apply class-aware batched NMS at 0.4 IoU and cap results at 25.

6. Threshold masks, restore source resolution, close small holes and simplify the largest external contour.

7. Return source-coordinate boxes, masks, polygons, thumbnails, mask area, centroid and a median foreground color.

Why another NMS step?

Region-based detectors already perform internal postprocessing. AuraSpace adds an explicit application-level filter/NMS stage. **IoU = area(A∩B) / area(A∪B)**. If two same-class boxes overlap strongly, the higher score is kept and the redundant box may be suppressed.

```
# Actual excerpts from detect_furniture(), inference.py
if int(lbl.item()) in FURNITURE_CLASS_IDS \
        and sc.item() >= DETECTION_SCORE_THRESHOLD:
    keep_mask[idx] = True
# ... filtered tensors ...
nms_indices = batched_nms(boxes, scores, labels, NMS_IOU_THRESHOLD)
nms_indices = nms_indices[:MAX_DETECTIONS]
```

Numerical example: two boxes each have area 100 and overlap by 60. Their union is 140, so IoU = 60/140 ≈ 0.429. For same-class outputs, that exceeds the extra 0.4 threshold. Class-aware suppression avoids applying this additional suppression across different class labels.

Outputs are compact approximations

The polygon helper keeps the largest external component and approximates it with an epsilon based on perimeter. It can lose small disconnected parts or holes. The raster mask artifact and transparent cutout preserve more detail for inspection. The foreground median color is a convenient display cue, not a reliable material or finish measurement.

Source trail: services/vision/app/inference.py; app/masks.py; apps/worker/src/processor.ts.

Why these models, and why not others?

The choices below explain fit to the current requirements. They are **engineering tradeoffs**, not results of a measured head-to-head experiment. A different model may win after testing on representative room photos.

Option	Benefit	Why it is not the primary server choice
Mask R-CNN	Labels, boxes and separate instance masks in one pretrained pipeline	Current choice; accepts CPU latency and restricted categories
YOLO detection	Attractive for fast box detection	A detection-only checkpoint lacks pixel masks. A segmentation variant is a credible future benchmark, not inherently unsuitable.
Faster R-CNN	Region-based labels and boxes	Lacks the parallel instance-mask output required for direct mask review
SSD / MobileNet	Smaller inference path, useful in browsers	Current browser fallback returns boxes rather than instance masks
SAM-style segmentation	Flexible prompt-based object masks	Requires an object proposal/prompt and category/identity logic for this inventory flow
Classical contours	Transparent and inexpensive	Texture boundaries do not provide robust category recognition
Training from scratch	Full control over categories/domain	Needs labeled data, training resources and independent evaluation

The browser uses a different detector

`src/services/vision.web.ts` lazily loads TensorFlow.js **COCO-SSD with lite_mobilenet_v2** when the server analysis path is unavailable/unconfigured. It uses an analysis canvas capped at 1280 pixels, up to 50 boxes and a 0.3 score cutoff, then maps boxes back to the source dimensions. Model download may require a connection on first use. [R5]

The web client can discover a connected vision service through `/api/health`; otherwise this local detector remains useful. Browser pixel processing and approximate surface fill also provide lightweight local functions. They do not duplicate native mask review, server job durability or metric depth.

Why two paths? The web offers a smaller browser runtime, while server inference provides instance masks and heavier models. Be precise: the Android app does not silently run the TensorFlow.js browser detector offline; native manual planning works offline.

Source trail: src/services/vision.web.ts; services/vision/app/inference.py; package.json. Browser model implementation: [R5].

Big-LaMa: learned image completion

Why use a learned inpainting model?

Removing a large sofa may expose a wide wall or repeating floor pattern. Local propagation alone has little information to build that appearance. LaMa uses fast Fourier convolution components to provide broad image context and was designed for large-mask completion. This motivates testing it for removal previews; it does not prove superior quality on this project's rooms. [R6]

The project loads a **Big-LaMa TorchScript export distributed by IOPaint**, pinned by SHA-256. Inputs are RGB image and binary mask tensors; output is checked for tensor shape and finite values. Loading and inference are guarded by a lock, and the configured adapter runs on CPU.



Recorded example, not a new benchmark: the smoke artifacts show couch detection and a Big-LaMa preview. The bottom-left panel reconstructs the polygon selection used in the smoke script, including 3 px expansion; it differs from the raster instance mask. Polygonization fills a hole around the left cat, so the preview illustrates an unwanted removal too. Outside-selection decoded pixels match exactly; this is execution evidence, not successful furniture-only completion or a quality benchmark.

Why not diffusion here? Diffusion-based inpainting is a credible future option, with potential tradeoffs in model/runtime cost, prompt control and repeatability. This repository already integrates a bounded image-plus-mask LaMa path; no comparative diffusion quality study has been run.

Source trail: services/vision/app/lama.py; model-manifest.json; .artifacts/model-smoke/*; reports/model-smoke.json. Research [R6]; export [R7].

Depth: useful evidence, limited certainty

Which model is actually used?

The configured learned model is **Depth Anything V2 Metric Indoor Small (Hypersim)**, from the pinned Hugging Face repository revision listed in the manifest. The loader checks all local model/configuration files and uses Transformers with local-only files and safetensors. A leftover `MiDaS_small` setting in config is not the implemented learned depth path.

Monocular means one RGB image is used. **Metric** means the output is intended to represent physical depth in meters, rather than an arbitrary relative ordering. Learned scale is still an estimate; real-room validation is required. The model does not supply missing camera calibration or prove the geometry behind furniture. [R8]

Raw depth and visualization are different artifacts

• The model's raw float output retains **meters** and is resized to source image dimensions.

• Display uses the 2nd/98th percentiles, normalization and a MAGMA color map. Its colors are not physical measurements.

• Transport encodes round(depth × 1000) in a **16-bit PNG**, so each stored unit is one millimeter, capped at 65.535 m.

• PNG preserves the integer map; rounding introduces up to about 0.5 mm quantization for in-range values. This is transport precision, not model accuracy.

```
# Actual core from encode_metric_depth(), depth.py
millimetres = np.clip(np.rint(values*1000), 0, 65535).astype(np.uint16)
ok, png = cv2.imencode('.png', millimetres)
```

What if no metric model is configured?

The service returns a clearly labeled **heuristic visualization** based on Sobel texture and vertical image position. It is neither learned nor measured depth. It must never be used as room dimensions. If optional depth fails during analysis, detection can still return with a warning.

Why not stereo or LiDAR? Stereo needs multiple calibrated views and correspondence; depth sensors need supported hardware. A monocular model works with ordinary photos, but trades physical certainty for accessibility. ARCore/sensor fusion belongs to future work.

Source trail: services/vision/app/depth.py; app/routes.py; services/vision/model-manifest.json. Upstream model family and metric variant: [R8].

Room geometry: what is real today?

Classical evidence that is implemented

The room helper converts to gray, blurs, runs fixed-threshold Canny and uses **probabilistic Hough lines**. It groups near-horizontal and near-vertical segments. Hough's line relation is **rho = x·cos(theta) + y·sin(theta)**: many aligned image points support the same line parameters.

Vanishing-point candidates come from extended line intersections and coarse clustering. Parallel directions in a 3D room can converge in a perspective image. However, furniture edges, lens distortion and clutter also produce intersections; a cluster is only a candidate.

The helper's coarse “boundary polygon” uses extreme horizontal/vertical image lines or falls back to the image rectangle. It is not a reliable floor/wall reconstruction. More importantly, the mobile worker consumes a separately labeled `room_proposal` with placeholder corners.

The four initial floor corners are placeholders

```
# Actual proposal values in analyze_room(), routes.py
"floorCornersPx": [
    [w*.15, h*.55], [w*.85, h*.55],
    [w*.98, h*.98], [w*.02, h*.98]
],
"source": "placeholder"
```

These values depend only on image dimensions. Users must correct them and enter known dimensions. They are not inferred from the Hough lines or learned depth. Current furniture dimensions likewise begin as category priors unless measured.

What about RANSAC plane fitting?

There is a RANSAC helper in `room_analysis.py`, but `/analyze` does not invoke it. It builds points from normalized image coordinates plus depth without a calibrated pinhole unprojection. It should be described as an experimental helper, not proof of a metric floor plane.

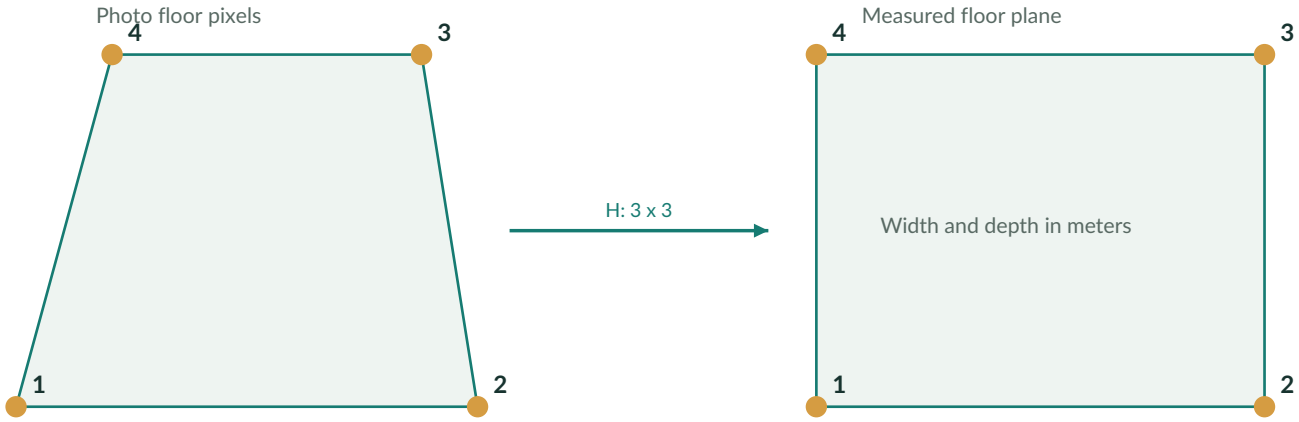
For a calibrated future implementation, a depth pixel with camera-axis depth Z could map to **X=(u−cx)Z/fx, Y=(v−cy)Z/fy**. Real intrinsics and depth conventions matter. Plane fitting must then identify the actual floor and transform camera coordinates into the scene frame. This future path is not the current reconstruction pipeline.

Source trail: services/vision/app/room_analysis.py; app/routes.py: analyze_room; apps/worker/src/processor.ts; docs/completion-roadmap.md.

Calibration with four floor points

A homography maps one plane to another

The review tool accepts four ordered image points and room width/depth. It solves a **3 × 3 homography H** mapping the visible floor plane to a rectangular metric floor reference. Four non-degenerate point pairs provide eight constraints after fixing one scale degree of freedom.



For photo point (x,y), compute **q = H·[x,y,1]ᵀ**, then world floor coordinates **(q0/q2, q1/q2)**. The target corner order is (0,0), (width,0), (width,depth), (0,depth).

```
// Actual formula from mapFloorPoint(), calibration.ts
const d = matrix[6] * x + matrix[7] * y + matrix[8];
const mapped: PointXZ = [
  (matrix[0] * x + matrix[1] * y + matrix[2]) / d,
  (matrix[3] * x + matrix[4] * y + matrix[5]) / d,
];
```

What the checks protect against

The solver requires four finite points and positive dimensions, rejects crossed/non-convex or collinear corner orders, and rejects nearly singular systems. The mapper rejects a near-zero denominator or non-finite result.

Limitations: the mapping is valid for the selected plane. A sofa arm or chair back is above the floor and cannot be measured by simply applying a floor homography to all its pixels. Incorrect point order or approximate dimensions create incorrect scale even when the algebra succeeds.

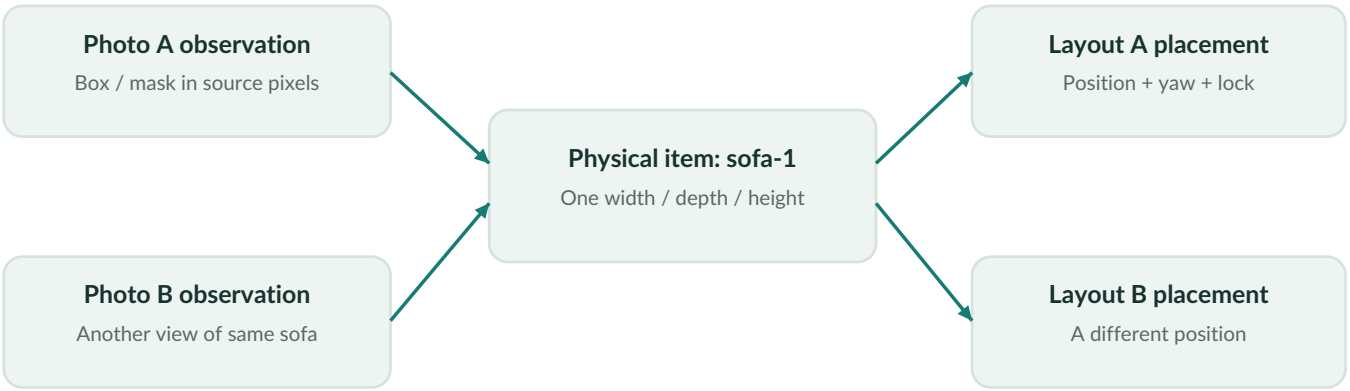
The stored `floorReview` records corners and the homography. This is a review/calibration primitive; current code does not turn it into a complete calibrated 3D reconstruction or automatically derive all furniture dimensions. A four-corner rectangle also does not represent every possible concave room boundary.

Source trail: packages/scene-domain/src/calibration.ts; schema.ts: floorReview; apps/mobile/src/features/capture/CalibrationReview.tsx.

Observation, physical item, placement

Why do we keep three separate concepts?

A detector sees an **observation in one image**. The real sofa is one **physical item**. A layout contains one **placement of that item**. Confusing these creates duplicated furniture, inconsistent dimensions and accidental changes across alternatives.



Entity	Stored information	Example
Observation	Capture ID, source-pixel box/mask, score, model/manual source, fixed/ignored/reviewed flags	Photo A sees the left side of a sofa
Physical item	Linked observations, category, dimensions with provenance, color, representation and movability	sofa-1 is 2.0 × 0.9 × 0.85 m
Placement	Physical item ID, floor-center X/Z, yaw radians and locked flag	sofa-1 at (1.5, 1.0) in Layout A

Users can link another photo to an existing item, merge duplicate identities or split an incorrect association. This is **reviewed multi-view identity**, not automatic geometric multi-view fusion or a trained re-identification model.

Changing the physical sofa width changes its representation in every native layout. Moving it changes only the active layout's placement. Returning an item to the tray removes its placement, not its physical inventory identity. A fixed fixture must retain the same locked pose across every layout.

Per-axis provenance matters

Each width, depth and height has a value, source, reviewed flag and optional evidence. Sources include user-measured, product-specification, category-prior, model-estimated and placeholder. The schema allows future sources; their presence does not mean every estimation mechanism is implemented. A sofa may have measured width and default height at the same time.

Source trail: packages/scene-domain/src/schema.ts; scene.ts: extract/link-observation/merge/split/fixed commands; tests/scene-v2.test.mjs.

What evidence do we actually have?

Execution evidence versus accuracy evidence

The recorded Mask R-CNN smoke run detects a **couch** in public COCO image 39769. Its report records about **3278 ms** processing on the development host. The analysis artifact has score 0.9097. Big-LaMa ran and the report records unchanged decoded pixels outside the mask. One successful image does not establish general accuracy or good hidden-surface completion.

The recorded depth smoke run produces a finite **480 × 640 meter-valued array**, with values approximately **1.157–1.825 m**, in **7.40 s** on that host. Its accuracy field is explicitly `not_evaluated`. Do not call these timing figures phone performance, a stable benchmark or ground-truth room dimensions.

What would a proper evaluation report?

Area	Appropriate measurements
Detector	Precision, recall and official box AP over a frozen independent test set; failures by category/viewpoint
Instance masks	Mask AP, IoU/Dice; thin structures and occlusion errors
Metric depth	MAE/RMSE and relative depth error using valid independent depth references
Dimensions	Absolute and relative error against tape/laser measurements; measured user inputs reported separately
Removal	Paired/synthetic masked-region measures, outside-mask equality, human quality and correction effort
Runtime	Cold/warm time, hardware, image size, memory and repeated runs

Precision = TP/(TP+FP); **recall = TP/(TP+FN)**. AP summarizes a precision-recall curve; COCO-style reporting uses defined IoU ranges and matching rules, not one convenient image or one threshold.

The repository's evaluation helper only computes greedy, category-aware **box precision/recall at IoU 0.5** for non-crowd furniture. It is not COCO mAP and does not evaluate masks or depth. The bundled evaluation report deliberately leaves unsupported metrics empty.

Independent split: select evaluation rooms without tuning repeatedly on their answers, retain annotations and hashes, and report failures. Synthetic unit tests and mocked detector contracts establish software behavior, not learned model accuracy.

Source trail: services/vision/reports/{model-smoke,depth-smoke,evaluation_report}.json; .artifacts/model-smoke/analysis.json; scripts/evaluate.py; docs/evaluation.md.

Computer-vision viva questions

Q1. What did your project train?

No custom model training is implemented. We integrate verified pretrained Mask R-CNN, Big-LaMa and a metric depth model. We contributed the source-coordinate review flow, scene/identity rules, planner and durable processing system.

Q2. Why Mask R-CNN instead of a normal object detector?

We need separate instance masks for cutouts and removal review as well as category boxes. A box-only detector cannot directly distinguish foreground from the background inside that rectangle. A modern segmentation alternative could be compared later.

Q3. Can the project detect every furniture category?

No. The configured server filter keeps chair, couch, potted plant, bed and dining table. Users can manually add other items. The editor's category list is broader than detector coverage.

Q4. Is metric depth the same as accurate dimensions?

No. It estimates distance to visible surfaces in meters, with potential error. Physical dimensions also depend on camera calibration, viewpoint, surface correspondence and boundaries. We preserve manual measurements and never measure from depth colors.

Q5. Does the project reconstruct a room from several photos?

It stores multiple views and lets users link observations to a single physical item. It does not currently fuse them into a verified metric room mesh. Proposed floor corners are placeholders pending review.

Q6. What is NMS and why use it?

It removes redundant overlapping same-class boxes by keeping higher-score predictions. We use class-aware batched NMS at 0.4 IoU after filtering. It is separate from Canny edge thinning.

Q7. Why no published accuracy percentage?

The repository contains execution smoke tests, not an independent quality benchmark. We can explain the evaluation plan and show honest recorded evidence. A confidence score and passing unit tests are not detection accuracy.

Q8. What is the hardest unsolved vision issue?

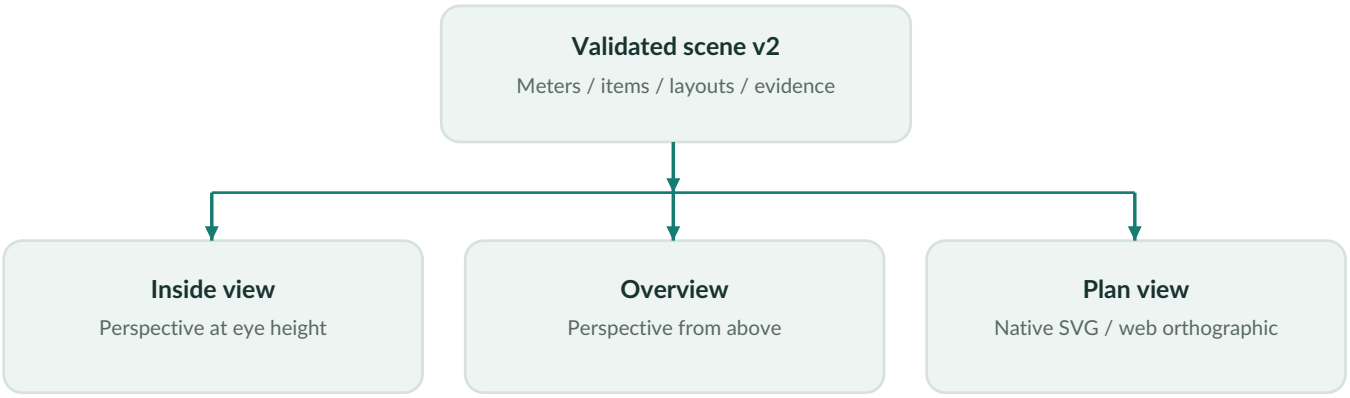
Reliable metric geometry from ordinary cluttered photos: scale ambiguity, occlusion, missing intrinsics, unusual furniture and unseen surfaces. Manual corrections and explicit provenance make the current assistant useful while those estimation problems remain open.

Source trail: services/vision/model-manifest.json; app/{inference,depth,routes}.py; packages/scene-domain/src/schema.ts; docs/implementation-audit.md.

03 / Interactive computer graphics

What makes this interactive graphics?

The system stores a geometric scene, projects it through cameras, renders surfaces and responds to user input by changing scene state. Users can select, place, move, rotate and inspect furniture while seeing spatial feedback. This is more than a static image or a set of before/after photographs.



Scene representation and coordinate system

The native **scene v2** document stores units in meters, a floor polygon in X/Z, room height, openings, colors, captures, observations, physical items and multiple layouts. Position is the furniture footprint's center: `floorCenterXM`, `floorCenterZM`. **Y points up**, and floor level is Y = 0.

Native placement yaw is stored in **radians around +Y**. The legacy web format stores clockwise plan rotation in degrees. The adapter uses negative yaw/degrees conversion so rendered footprints and geometry tests agree. Saying every module uses the same rotation unit would be wrong.

One scene, three views

Inside gives a human-height perspective; overview gives a raised perspective; plan provides a precise top view. The native plan is drawn with **react-native-svg**, whereas the web plan uses an **orthographic Three.js camera**. Switching the camera or representation does not create a different furniture layout.

The shared domain engine validates scene commands and identity/placement relationships. The assets package draws furniture. The API contracts define network inputs. This separation lets geometric rules be tested without a camera or GPU.

Important boundary: shared packages and adapters do not make the legacy web and native persistence formats identical. Native alternatives share item dimensions; web legacy layout snapshots retain their existing format.

Source trail: packages/scene-domain/src/{schema,scene}.ts; apps/mobile/src/features/editor/{NativeScene,PlanView}.tsx; src/components/scene/SceneWorld.tsx.

Procedural furniture and transforms

Why generate furniture from geometric parts?

The shared `FurnitureModel` builds sofas, chairs, beds, tables, cabinets, plants, rugs and custom boxes from shapes such as rounded boxes, cylinders and planes. Dimensions control the generated geometry. A sofa can have a seat, back, arms and feet positioned relative to its width/depth/height.

This approach offers a small, editable representation without requiring a unique scanned mesh for every photograph. It is useful for testing layout and approximate fit. It does not reproduce the exact real object's shape, upholstery or hidden geometry. A photo cutout is a 2D artifact, not an automatically reconstructed 3D mesh.

Local coordinates become world coordinates

Conceptually, a vertex passes through **clip = Projection × View × Model × localVertex**. The model transform places an object in the room; the view transform expresses it relative to the camera; projection maps it to screen space.

Native furniture groups use footprint-center translation and Y-axis yaw. For a right-handed +Y rotation, a floor point follows **x' = cx + x·cos(yaw) + z·sin(yaw)** and **z' = cz − x·sin(yaw) + z·cos(yaw)**. The legacy clockwise angle uses the corresponding negative-yaw convention.

```
// Actual placement structure, NativeScene.tsx (excerpt)
<group position={[pose.floorCenterXM, 0, pose.floorCenterZM]}>
  <FurnitureModel item={legacyItem(item, pose)} />
</group>
// legacyItem supplies dimensions and converted rotation to the model.
```

Materials, lights and useful approximations

The assets use standard lit materials with color and roughness. Native rendering includes ambient, hemisphere and directional light plus shadow mapping. A translucent preview shows a candidate placement; selected footprints use warning-aware colors. Cutaway walls in overview make the interior visible.

Why not photorealistic meshes or NeRF/Gaussian reconstruction? They can improve appearance, but editable dimensions, object identity, fit constraints and reliable mobile performance still need separate engineering. Future approved glTF assets could improve fidelity; current schema support for an approved-model representation is not evidence of a complete model library/import flow.

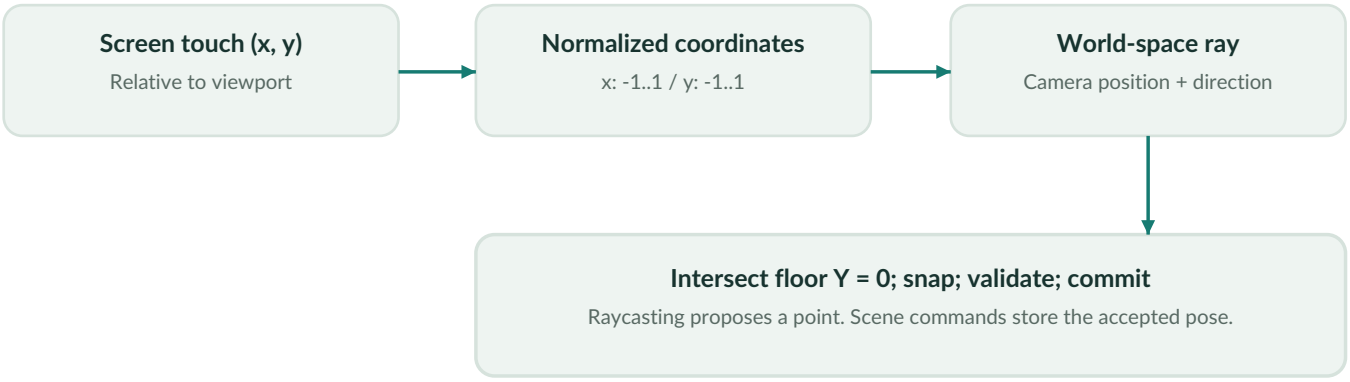
Source trail: packages/scene-assets/src/FurnitureModel.tsx; packages/scene-domain/src/scene.ts: legacyItem; apps/mobile/src/features/editor/NativeScene.tsx.

Cameras and touch-to-world mapping

Perspective versus orthographic

A perspective camera makes farther objects look smaller. Native inside uses a **68° field of view**; overview uses **38°**. Its camera setup uses near 0.03 m and far 200 m. Eye-height placement, aspect ratio and look direction affect perceived scale and navigation.

An orthographic camera removes distance-based scaling, which is useful for a plan. The web implements that camera directly; native SVG maps a room viewport to screen coordinates with linear scaling. Both display the same floor-space layout through different rendering mechanisms.



How does a tap become a floor coordinate?

Normalize viewport-relative touch position: **x_ndc = 2x/W − 1**, **y_ndc = 1 − 2y/H**. A raycaster creates a world ray through that screen point. Intersect it with the floor plane Y = 0; use hit.x and hit.z as a candidate.

```
// Actual ray construction, NativeScene.tsx (excerpt)
raycaster.setFromCamera(
  { x: (2*x)/size.width - 1, y: 1-(2*y)/size.height } as any,
  camera,
);
const hit = raycaster.ray.intersectPlane(FLOOR, new Vector3());
return hit ? [hit.x, hit.z] : null;
```

If r(t)=o+t·d, the floor hit satisfies **t=−oy/dy**, when dy is nonzero and the intersection is usable. Library ray-plane logic handles this calculation. A parallel ray has no floor hit; an interface should preserve the draft rather than invent coordinates. [R9]

The editor optionally snaps X/Z to **0.10 m**: round(value × 10)/10. That is a placement grid, not a measurement accuracy guarantee. Hit testing can select a mesh, while floor raycasting proposes a location; neither operation alone validates collisions.

Source trail: apps/mobile/src/features/editor/NativeScene.tsx; apps/mobile/app/room/[id].tsx; PlanView.tsx; src/components/scene/SceneWorld.tsx. Raycaster [R9].

Interaction, commands and state

A deliberate placement flow

Open **Add furniture** → choose a tray item → tap the floor or use supported tray dragging → preview candidate → confirm placement. Selecting or dragging a placed object shows compact actions. **Details** opens measurements and properties explicitly. Tap alternatives make long-press interactions less restrictive.

The native editor uses a Zustand store and shared scene commands. During dragging, transient preview state can update feedback; committed commands validate and store the result. This separates a visual candidate from the persisted scene, so cancellation can leave the previous accepted placement intact.

Action	Domain-level effect
Add / extract	Create or link physical inventory; preserve source observation identity
Place / unplace	Add/remove an active-layout pose; unplacing returns the item to the tray
Change dimensions	Update the physical item shared across native layouts
Lock / fixed	Lock a pose; fixed items enforce the same pose across layouts
Save alternative	Keep a separate placement arrangement referencing existing items
Undo / redo	Restore scene content with a fresh increasing revision

Why command validation?

The schema rejects broken references, duplicate IDs, invalid dimensions, invalid floor boundaries and inconsistent fixed fixtures. A shared command engine keeps these rules independent of whichever UI initiated the change. Zod also validates external and stored data: TypeScript alone does not validate arbitrary JSON at runtime.

Undo/redo retains bounded snapshots (up to 50 in the shared history logic). Undo changes content but increments revision; rolling a revision number backward would confuse server concurrency checks. History is not a complete audit ledger or a guaranteed persistent undo timeline across process restarts.

Performance and fallback

The native canvas uses **demand rendering** and invalidates when needed, reducing unnecessary continuous redraws in a mostly static editor. Geometry memoization and disposal avoid recreating resources unnecessarily. These choices still require physical-device profiling. If 3D cannot initialize, the editable plan remains available.

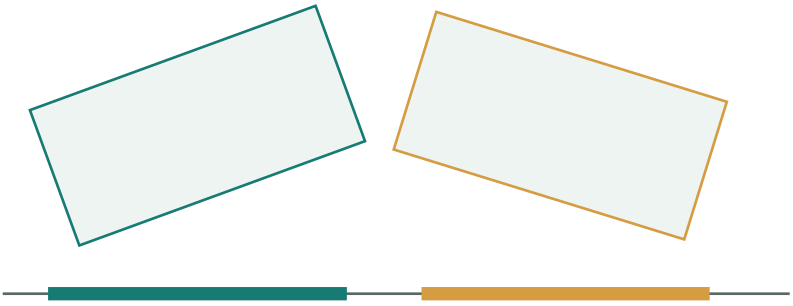
Feedback versus blocking: domain validity and locks constrain commands; spatial rules also produce warnings. A warning should not be described as a general-purpose physics solver that automatically moves furniture into a valid arrangement.

Source trail: apps/mobile/src/state/studio.ts; packages/scene-domain/src/scene.ts; schema.ts; apps/mobile/src/features/editor/NativeScene.tsx.

Collision, containment and fit

Why simple unrotated boxes are insufficient

A sofa rotated 45° occupies a different floor footprint. Axis-aligned bounding boxes can report overlap even when rotated rectangles do not touch. AuraSpace computes rotated corners and uses the **Separating Axis Theorem (SAT)** on their floor footprints.



A separating projection axis means no overlap.

SAT for rotated footprints

Project corners onto edge normals. One separated interval is enough. Touching edges are allowed.

For each candidate edge normal n, project every corner p using **s = p·n**. If either projection interval ends before the other begins, the objects are separated. If there is no separating axis, they overlap. Rectangles have two independent edge directions each; the implementation loops edges of both polygons, including redundant parallel directions.

```
// Actual interval test, legacy/geometry.ts (excerpt)
if (
  Math.max(...pa) <= Math.min(...pb) + EPSILON ||
  Math.max(...pb) <= Math.min(...pa) + EPSILON
) return false; // one separating axis: no overlap
```

More than pairwise overlap

The native floor can be concave. Merely checking the furniture's center or four corners may miss an edge crossing a concave notch. `footprintContained` checks corners, edge midpoints and crossings with the room boundary. Other warnings cover ceiling height and overlap with conservative inward-door clearance footprints.

Furniture collision uses an approximate rectangular footprint, not full triangle-mesh intersections or rigid-body dynamics. Rugs are treated as floor decoration for overlap rules. A table and a chair may have physically compatible interlocking geometry that rectangle tests flag conservatively.

Occupied/free area is withheld when overlap or boundary-invalid placements make simple area summation misleading. It is not the exact geometric union area of arbitrarily overlapping solids. Door checks use conservative squares rather than precise swept door arcs, and do not certify regulatory compliance.

Source trail: packages/scene-domain/src/legacy/geometry.ts; spatial.ts: footprintContained, placementWarnings, roomFeedback, openingClearance.

Walking and circulation feedback

Two related but different checks

Guarded walking: to move the inside camera, the app checks a proposed path from the current X/Z position to the target. It samples the segment at roughly 5 cm intervals and checks whether a standing footprint with a default 0.18 m margin remains in the room and outside obstacle footprints.

Circulation: for start/end floor points, the domain helper discretizes the room and uses breadth-first search over a conservative occupancy grid. Defaults are **0.70 m passage preference** and **0.15 m grid step**. Four-neighbor moves avoid diagonal corner cutting. The helper limits the grid to 100,000 cells.

Choice	Benefit	Tradeoff
Sampled straight-line walking	Cheap check for a proposed local move	Does not automatically route around all obstacles
BFS grid	Simple reachability and shortest grid-step path with equal-cost moves	Coarse grid, approximate clearance and repeated obstacle tests
A*	Could reduce explored nodes using a distance heuristic	Future optimization; same clearance model still needs validation
Navigation mesh	Could model larger continuous navigable regions	More geometry generation and update complexity

Complexity and practical limits

SAT per pair is constant work for four-corner footprints; checking all furniture pairs is approximately $O(n^2)$. A grid search is O(V+E) for graph traversal, but each occupancy/clearance test can also inspect furniture and room geometry. Spatial indexing and cached occupancy are future optimizations for large scenes.

A 60 FPS display has about **16.7 ms per frame**, but the project does not have measured phone frame-rate results. Processing images for seconds on a worker is separate from the graphics frame budget. Neither the recorded model timing nor a browser viewport test establishes native touch performance.

What should we claim in the viva?

“We provide approximate spatial warnings and conservative circulation assistance. The path depends on chosen clearance and grid resolution. It does not establish accessibility, fire safety or building-code compliance, and it is not a full human-body motion simulation.”

The current scene is editable even when a warning appears. A future arrangement optimizer could search layouts and score these constraints; the present planner remains user-driven.

Source trail: packages/scene-domain/src/spatial.ts: canStand, walkPathClear, circulation; apps/mobile/src/features/editor/NativeScene.tsx; docs/implementation-audit.md.

Why this technology stack?

These choices balance a mobile capture workflow, a responsive editor and a Python model ecosystem. They are defensible tradeoffs; the repository does not prove that every alternative is inferior.

Technology	Why it is used here	Alternative and tradeoff
TypeScript + pnpm workspace	Shared domain/contracts/assets across JavaScript apps; typed commands	Separate repos/languages increase contract duplication; runtime validation still needed
React Native + Expo	Android camera/files/SQLite/GL/sharing plus React UI	Native Kotlin offers tighter platform control but requires different UI/shared-code work
Next.js + React	Web editor, routes and server vision proxy in one app	Vite plus separate backend is viable; another deployment/configuration split
Three.js + React Three Fiber	Cameras, geometry, materials and raycasting expressed in React; native GL integration	Raw WebGL/OpenGL needs more graphics plumbing; Unity/Godot add a different engine/toolchain
SVG	Clear dimensioned plan/export and native plan fallback	Raster plan loses vector editability; 3D alone adds unnecessary dependency for top view
Zustand	Shared editor state/actions with relatively little boilerplate	Redux is viable but more structure; component state alone complicates cross-panel updates
Zod + Pydantic	Validate scene/HTTP JSON in TypeScript and Python	Static types alone cannot reject invalid incoming data
Fastify + FastAPI	TypeScript orchestration API; Python image/model runtime	A single Python service could simplify deployment but reduce shared TS integration
PyTorch / torchvision / Transformers	Existing verified detection and depth implementations/checkpoints	TensorFlow is viable; rewriting models is unnecessary for this application
PostgreSQL + Prisma	Durable related records, revisions, ownership, migrations and typed DB access	MongoDB is viable for documents, but relations/constraints would need different design
SQLite + files	Local projects work offline; structured metadata separate from binary photos	Cloud-only persistence makes manual planning network-dependent
Redis + BullMQ	Background scheduling, retries and leases in Node	DB polling, RabbitMQ or managed queues are viable; see page 33
Docker Compose	Start compatible services with defined networking/volumes	Manual installs can work but need more per-machine setup

Version boundary: web and mobile resolve different React/Three.js generations. Root and app manifests are the authority for each package. The run guide suggests Node 24+, while current Dockerfiles use Node 22; record the actual environment when reproducing a run rather than claiming one universal runtime version.

Source trail: package.json; apps/{mobile,api,worker}/package.json; packages/*/package.json; Dockerfiles; RUNNING.md. R3F background: [R10].

Why did we use Docker?

The concrete problem Docker helps solve

AuraSpace has Node services, a Python vision runtime, PostgreSQL, Redis and local model files. Installing and coordinating compatible dependencies manually on every machine is error-prone. Docker packages service environments; Compose describes how to start and connect them. A container runs as an isolated process environment, using the host kernel rather than booting a separate full guest OS for each service. [R11]

Compose component	Responsibility
database	PostgreSQL 16 with health check and durable named volume
redis	Redis 7.4, password, append-only persistence and named volume
migrate	One-shot Prisma migration step after the database is healthy
api	Fastify service on container port 4000
worker	Same backend image, different command to consume jobs
vision	Python runtime, CPU PyTorch, verified model directory mounted read-only
web	Next.js build and web companion; talks to vision internally

Images, containers, volumes and networks

An **image** is the service template; a **container** is a running instance. A **volume** keeps database/queue data beyond a container's lifecycle. Internal DNS names such as `database`, `redis` and `vision` let services talk without hardcoding changing container IPs.

Inside the API container, `localhost` refers to that API container, not Redis. In the standard Android emulator, `10.0.2.2` reaches the host; a physical phone needs the host's reachable LAN address. These addresses solve different networking problems.

```
# From the repository root after configuring .env
docker compose -p auraspace up --build -d
# Inspect service state and recent worker/vision output
docker compose -p auraspace ps
docker compose -p auraspace logs --tail=80 worker vision
```

Why not just use a Python virtual environment?

A venv handles Python dependencies; it does not manage PostgreSQL, Redis or Node. Local non-container execution is supported and can be convenient during development. Containers improve packaging and startup consistency; they do not automatically supply model accuracy, secure production hosting, backups or complete application readiness. The declared startup dependencies are not a full readiness check for every service.

Supabase Storage is external in the current configuration; Compose does not launch a complete local Supabase platform. Model weights are mounted rather than bundled into the image.

Source trail: docker-compose.yml; Dockerfile; infra/backend.Dockerfile; services/vision/Dockerfile; RUNNING.md. Container concepts [R11].

Why Redis, and why BullMQ?

Redis is used as the job queue substrate

The API should acknowledge analysis quickly and let the user keep editing. Running model inference synchronously in the request would tie the operation to request duration and client connectivity. Redis stores BullMQ's scheduling state; a worker consumes jobs, records stages and invokes Python.

Redis is not being used as a general-purpose image cache or the only durable database. Queue messages contain a job ID. Project records, immutable input snapshots and job intent live in PostgreSQL; photo/mask/preview bytes live in object storage.

```
// Actual defaultJobOptions, packages/backend/src/queue.ts
attempts: 3,
backoff: { type: 'exponential', delay: 2000 },
removeOnComplete: { age: 86400, count: 500 },
removeOnFail: { age: 604800, count: 500 },
```

The worker uses a 300,000 ms lock duration and supports stalled-job handling. BullMQ's processing lock is a **lease**, not a promise that a crashed machine can never run work twice. Expiring ownership, worker recovery and retries require idempotent handling. Exponential retry settings control retry spacing. [R12]

Why add PostgreSQL durability as well?

Redis enqueue can fail after the API saves a job. The API keeps that intent and returns 202; the worker periodically scans old queued records and re-enqueues missing jobs. The Compose Redis also enables append-only persistence on a volume. AOF helps recover Redis state but does not replace business records or guarantee zero data loss under every failure. [R13]

Alternative	When it could be reasonable	Current tradeoff
In-memory array	Single-process disposable demo	Lost on restart, little coordination or recovery
PostgreSQL queue/polling	Simplify infrastructure for modest load	Must implement locking, scheduling, retries and job lifecycle carefully
RabbitMQ	Rich broker routing and delivery patterns	Another broker and Node integration model; not required by current flow
Managed queue	Hosted elasticity/operations	Cloud dependence and provider-specific behavior

Best viva answer: “Redis/BullMQ schedules expensive background processing; PostgreSQL remains the durable source of intent. This split keeps interaction responsive while making retry and recovery explicit.”

Source trail: packages/backend/src/queue.ts; apps/worker/src/index.ts; apps/api/src/routes/jobs.ts; docker-compose.yml. Retry [R12], persistence [R13].

A durable job from request to review



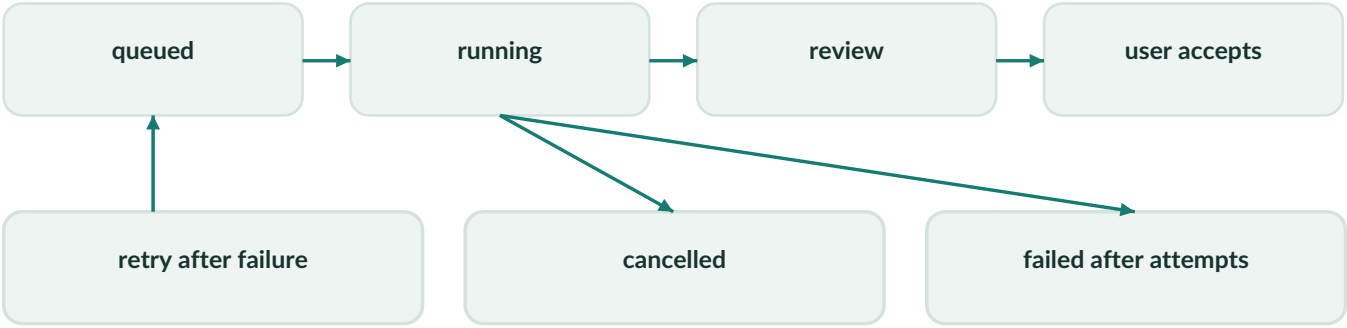
Recovery scans persisted queued intent when Redis enqueue is temporarily unavailable.

The request is tied to a specific input

The API validates ownership, capture readiness and **inputRevision**. It computes a canonical input hash and enforces uniqueness of the project's idempotency key. Repeating the same key/input returns the same job; reusing that key for different input produces a conflict. The stored job includes a snapshot of the scene and mask revisions.

Example: a user starts analysis at revision 12, then moves the sofa and enters its width at revision 13. A late revision-12 result is stored as a proposal. The worker does not replace the active revision-13 scene. Explicit acceptance merges useful observations while preserving reviewed work.

What happens while it runs?

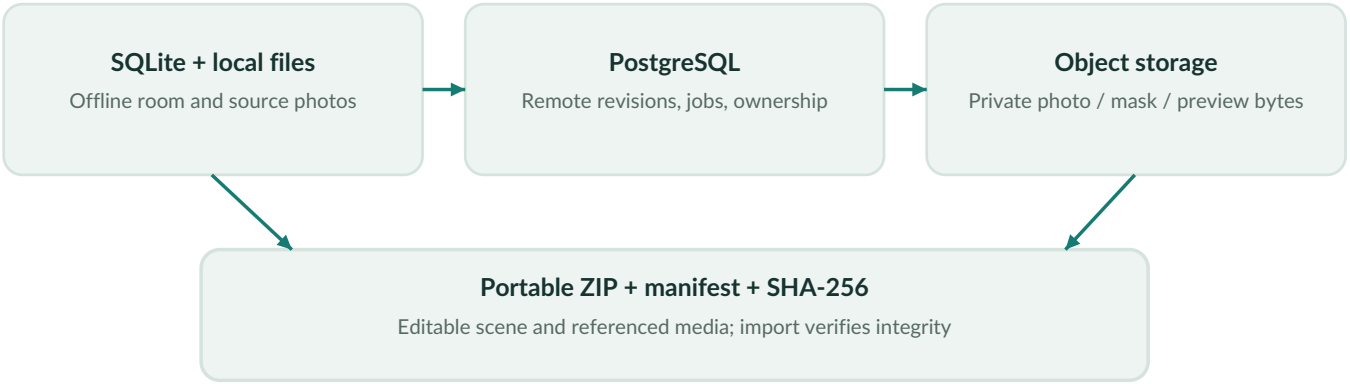


The processor checks cancellation/deletion between stages and before artifact writes. It verifies source byte count and SHA-256, validates image orientation/dimensions, calls the chosen operation and stores deterministic result artifacts. On success, the database job enters **review** with stage `complete`; that means processing finished, not that the user has accepted it.

Cancellation is cooperative: it can stop further stages but does not interrupt every running Python kernel instantly. Retries do not promise exactly-once execution; deterministic artifact IDs and idempotent writes reduce duplicates. Crash-after-upload, cancel races and broader machine-loss recovery still need fault-injection evidence.

Source trail: apps/api/src/routes/jobs.ts; apps/worker/src/{index,processor}.ts; packages/contracts/src/index.ts; docs/completion-roadmap.md.

Storage, offline work and synchronization



Why use more than one storage system?

SQLite stores validated native project documents, local media records, jobs, preferences, per-server revision links and an operation outbox. WAL supports the local database's read/write behavior. App document files hold originals and downloaded artifacts; relative paths make records less dependent on a temporary picker URL.

PostgreSQL stores remote projects, owners, revisions, capture/upload metadata, artifacts and durable jobs. Although scene content is JSON, relational records still support lifecycle and ownership checks. **Private object storage** holds the actual photo, mask, depth and preview bytes.

The current `MediaStore` calls the **Supabase Storage REST API directly**, verifies the bucket is private and issues temporary access URLs (300 seconds). Generic S3-compatible storage is an architectural option mentioned in some notes, but the active adapter is Supabase-specific; installing AWS SDK packages does not mean AWS S3 is currently used.

What does “offline” mean?

Manual planning and already-saved media work locally. Native cloud analysis requires the configured backend. The operation outbox retries supported pending operations on recovery; it is not an offline version of the learned models or an automatic collaborative merge engine.

Remote scene writes use **compare-and-swap**: an expected revision must match the server. A stale write returns conflict instead of silently overwriting someone else's scene. Native revision links are scoped by endpoint, and copying/importing remaps capture/media identities.

Export is portable; synchronization is still bounded

Native ZIP includes the editable scene, referenced media and a manifest with SHA-256 records. SVG/view sharing serves visual communication. Web JSON/SVG/PNG exports use the legacy browser format. A screenshot cannot replace an editable project.

Full cloud project browsing/download/restore and independent multi-device conflict trials remain roadmap work. Do not describe the current app as seamless real-time cross-client collaboration.

Source trail: apps/mobile/src/storage/{database,media}.ts; services/{sync,project-sync,sharing}.ts; packages/backend/src/media.ts; apps/api/prisma/schema.prisma.

API, ownership and data integrity

What crosses each boundary?

The public API orchestrates user/project actions. Python provides bounded internal image/model endpoints. Contracts and schemas reject malformed values and broken references. A private Python service is not a replacement for user authorization at the API.

Endpoint family	Purpose
/v1/projects and /:id/scene	List/create projects; fetch/update a scene with revision checks; delete project
/v1/projects/:id/captures	Create capture metadata before source upload
/v1/captures/:id/uploads	Prepare upload metadata; transfer bytes through upload route; finalize capture
/v1/projects/:id/jobs	Persist and enqueue analyze, process or inpaint requests
/v1/jobs/:id and /cancel	Poll durable status and request cooperative cancellation
/v1/artifacts/:id/access or /content	Owner-checked temporary access or content retrieval
Internal /validate, /analyze, /process, /inpaint, /depth	Python quality checks, detection, image operations, removal and depth

Authentication is not authorization

Authentication identifies the caller. Authorization checks that caller owns the requested project and its captures/jobs/artifacts. Guest sessions use random bearer credentials with server-side stored hashes. Optional OIDC/JWT verification checks issuer, audience and signature; the native auth flow supports PKCE. Live provider end-to-end validation remains open.

Service-role storage credentials stay server-side. Private bucket objects are addressed by durable IDs; signed URLs expire. “Private” does not mean objects are automatically end-to-end encrypted or impossible to leak after someone shares an access URL.

Integrity, validation and deletion

SHA-256 detects unexpected byte changes in uploads, downloads and exports; it is a hash, not encryption. Image dimension checks protect coordinate alignment. Bounds on payloads, pixels, masks and arrays constrain processing input. API errors are mapped into actionable responses; rate limiting is configured.

Project deletion tombstones the remote record, blocks access and lets worker cleanup remove associated artifacts. Backup retention, deployed TLS, disaster recovery and complete production security tests are additional operational work. No secrets or real credentials are included in this handbook.

Source trail: apps/api/src/routes/*.ts; app.ts; packages/backend/src/{auth,media}.ts; packages/contracts/src/index.ts; apps/mobile/src/services/session.ts.

Worked example: follow one sofa

Scenario

Assume a rectangular room measured **5.0 × 4.0 m**, height **2.8 m**. Two photos show the same sofa. The user measures its width as **2.0 m**, depth **0.9 m**, height **0.85 m**. These are example inputs for explanation, not reported measurements from an evaluated room.

1. **Capture:** import both photos. The app saves originals and bounded analysis copies; orientation and coordinate mapping are retained. Detection can run later while the local room remains editable.

2. **Vision:** Mask R-CNN proposes a couch observation in each image. Each has its own pixel box, mask and score. No box width is directly interpreted as 2.0 m.

3. **Review:** correct masks/category if needed and link both observations to physical item sofa-1. Otherwise two views could accidentally become two sofas. Mark actual fixtures fixed and false detections ignored.

4. **Measurements:** enter the three measured sofa axes with their provenance; keep defaults on any axis that has not been measured. Correct the floor corners if using calibration and retain the actual room dimensions.

5. **Preview:** select the movable sofa observations for removal. The worker checks mask revisions and creates separate inpainting previews. The original photographs remain intact.

6. **Graphics:** choose sofa-1 from the tray and place it at floor center **(1.5, 1.0)** with yaw 0. Its axis-aligned footprint spans X **0.5–2.5 m**, Z **0.55–1.45 m**. Confirm placement.

7. **Feedback:** rotate, move and inspect boundary, overlap and door warnings. A 90° yaw exchanges the footprint directions. A rendered arm or foot is still only part of the procedural approximation.

8. **Alternatives:** save Layout A, create another placement arrangement in Layout B, and move sofa-1 there. Updating its width updates both native layouts because they refer to the same item.

9. **Persistence:** edits save locally. When syncing, expected revisions protect the server copy. A delayed model proposal cannot automatically undo the measured width or recent placement.

10. **Export:** share a visual SVG/view and save a portable ZIP when editable scene and photos must travel together.

This example connects all three subjects: pixel masks support review/removal; vision supplies object proposals; graphics supports measurable placement. The infrastructure makes the workflow recoverable rather than tying it to one long HTTP request.

Source trail: apps/mobile/src/features/capture/*; packages/scene-domain/src/{schema,scene,spatial}.ts; apps/worker/src/processor.ts; services/sharing.ts.

What works, and what remains uncertain?

Be specific about implementation status

Area	Current evidence	Remaining uncertainty
Android app	Built/installable preview; native GL and SVG views, placement/persistence exercised in emulator	Real-phone camera, permissions, touch/accessibility, FPS, memory and thermals
Vision models	Verified local weights and real execution smoke artifacts	Independent room-domain precision/recall, mask quality and depth errors
Room shell	Editable boundary/openings/height and floor homography review	Automatic corners are placeholders; accurate automatic reconstruction unproven
Removal	Big-LaMa and labeled classical fallback; preserved outside-mask decoded pixels	Correct hidden appearance and quality/correction effort on independent rooms
Layout reasoning	Footprint SAT, concave containment, doorway warnings and grid circulation	Conservative geometry, no exact door arc or code certification
Backend	Durable jobs, CAS, private access and recorded integration verification	Sustained load, full machine-loss fault tests, deployed monitoring/backups
Sync	Server-specific revision links and conflict protection	Complete cloud restore UI and live multi-device conflict acceptance
Web	Useful browser-local companion and optional model service	Not the same persisted native workspace or automatic cross-client cloud sync

Common failure stories and recovery

No detection: let the user outline/add an item manually. **Incorrect instance:** review category/mask and link or split identity. **Poor inpaint:** refine selection, retain the original and continue with the procedural room. **Depth unavailable:** keep detection/manual dimensions.

No service/network: local editing remains available; pending supported operations retry. **Stale scene write:** preserve the draft and handle conflict or save a separate copy. **3D initialization failure:** use the plan view. **Storage failure:** present the error and retry/export rather than claiming autosave succeeded.

Some older documentation contains stale statements, including “no native build.” The later implementation audit, actual mobile source and APK verification are stronger evidence for current status. A source comment or an original plan is not proof a feature shipped.

Source trail: docs/implementation-audit.md; docs/completion-roadmap.md; artifacts/verification.txt; docs/evaluation.md; current source modules.

Future improvements, in sensible order

First: make the existing preview dependable

1. **Cloud restore and conflict workflow.** Add paginated browsing, scene/media download with hashes, resumable incomplete downloads and explicit preserve-both decisions. Acceptance: two independent clients can edit, conflict, restore and reopen without losing drafts.

2. **Fault injection.** Kill a worker, interrupt uploads, disconnect Redis/API, cancel during processing and simulate low storage. Acceptance: no lost measured values, no stale overwrite, understandable retry state and recoverable partial artifacts.

3. **Physical Android validation.** Test an older and a current ARM64 phone: real camera/EXIF variants, permissions, large text, touch, app background/restart, 20-item normal use and 50-item stress. Record frame times, memory and thermals.

Then: improve measured scene understanding

4. **Real geometry proposals.** Obtain intrinsics, define depth convention, unproject metric depth, robustly fit structural planes/lines and propose floor/wall/opening geometry with provenance and abstention. Acceptance: independently measured boundary/dimension error plus correction effort.

5. **Domain model evaluation and adaptation.** Freeze annotations, compare Mask R-CNN with suitable segmentation alternatives, evaluate difficult local furniture and consider fine-tuning only after a dataset exists. Measure box/mask AP, depth error and failures.

6. **Removal quality and usability.** Preserve mask holes/components through review and removal instead of losing them in polygon simplification. Evaluate paired/synthetic cases and real-room ratings; compare methods and measure correction effort. Keep generated imagery labeled.

Later extensions

7. **Deployment/accounts.** Validate live OIDC, hosted HTTPS, private storage, secrets, backups/restore, monitoring, retention and load limits. Acceptance includes cross-user access tests and disaster recovery evidence.

8. **Device scanning and richer graphics.** Add ARCore where supported, calibrated coordinate conversion, multi-view fusion and AR placement. Add approved glTF furniture, performance profiling, LOD/instancing or spatial indexing when measurements justify them.

9. **Arrangement search.** Optimize for overlap, door clearance, circulation and user preferences; expose suggested alternatives with explanations. This is a future optimizer, not a current AI layout designer.

Priority principle: improve data validity and measurable reliability before making stronger reconstruction claims. Each extension needs an independent acceptance test; an attractive demo is not enough.

Source trail: docs/completion-roadmap.md; docs/implementation-audit.md; PLAN.md for long-term intentions, not completion claims.

Reproduce, verify and demonstrate

The practical startup sequence

Use `RUNNING.md` as the environment authority. Install workspace packages and generate Prisma, configure root `.env` with the required private storage settings, then start dependencies/migrations and app services. Local Compose is the compact route; no real credentials belong in screenshots or a viva report.

```
# Preparation and local service startup
pnpm install
pnpm db:generate
pnpm check:env
docker compose -p auraspace up --build -d
# In separate terminals if using the development clients:
pnpm dev
pnpm dev:android
```

An Android development client or the built preview APK is needed; native GL modules are not equivalent to opening the web page. For emulator API access use `http://10.0.2.2:4000`; a phone needs a reachable LAN address and deliberate binding. Production requires HTTPS.

Verification commands and what they establish

```
pnpm typecheck:all
pnpm test
PYTHONPATH=services/vision python3 -m unittest discover -s services/vision/tests
node --env-file=.env --import tsx scripts/verify-platform.ts
node --env-file=.env --import tsx scripts/verify-processing.ts
```

The repository records **42 JavaScript cases and 19 Python tests**, plus platform/processing/browser and Android emulator verification. The unit suites cover geometry/domain/pixel behavior and contracts; real model smoke and platform checks are separate. The guide-generation run rechecked the JavaScript and Python unit suites; it did not repeat hosted/phone acceptance tests.

A coherent 5–7 minute demonstration

Open an example room → switch three views → select tray item and confirm → move/rotate and show a warning → undo → inspect dimensions and their source → import/analyze a photo → review a mask and identity → compare removal preview with original → save alternative → reopen → export.

Prepare already-recorded model outputs in case live inference is slow, and label them as recorded. When showing depth, explain meters versus color visualization. Say explicitly that floor proposals need correction and that physical-phone and accuracy gates remain open.

Source trail: RUNNING.md; README.md; docs/demo-script.md; artifacts/verification.txt; scripts/verify-*.ts; tests/*; services/vision/tests/*.

Graphics and infrastructure viva

Q1. Why Three.js and React Three Fiber?

Three.js provides scene/camera/material/raycasting primitives. Fiber connects those objects to React state and components, including the native GL path. We retain control over a custom planner without building all graphics plumbing from raw APIs. [R10]

Q2. How does dragging a 3D object work?

Touch coordinates become a camera ray; the floor intersection supplies X/Z. The editor previews, optionally snaps, validates and commits a pose. The model keeps physical dimensions; moving it changes placement rather than photo pixels.

Q3. Why SAT instead of a physics engine?

The task primarily needs rotated floor-footprint warnings, not forces, falling or rigid-body motion. SAT is simple and testable for rectangles. It is approximate for curved or interlocking real furniture.

Q4. Why Docker and Redis together?

They solve different problems. Docker packages and connects services; Redis/BullMQ schedules background inference with retries and leases. Neither replaces PostgreSQL durability or private media storage.

Q5. What if Redis goes down after the job is requested?

The API has already persisted the job snapshot. Its enqueue may fail, but recovery scans queued database records and retries enqueueing. A running operation still needs careful failure handling; we do not promise exactly-once execution.

Q6. Why PostgreSQL, SQLite and Supabase Storage?

PostgreSQL keeps durable server records and revisions; SQLite keeps the mobile project usable offline; Supabase Storage stores large private binary assets. Different locations have different responsibilities rather than three copies of one interchangeable database.

Q7. How do you avoid late AI results destroying edits?

Jobs capture input revision and scene snapshot. Workers save proposals rather than changing the live scene. Users review results explicitly, and remote scene updates use revision-based compare-and-swap.

Q8. Is this AR, a CAD tool, or an automatic interior designer?

It is an assisted room/furniture planner with procedural graphics and reviewed image analysis. Current output is not AR placement, a certified CAD drawing, a verified scan or an automatic layout optimizer. Those extensions require separate implementation and evaluation.

Source trail: apps/mobile/src/features/editor/NativeScene.tsx; packages/scene-domain/src/spatial.ts; apps/worker/src/processor.ts; packages/backend/src/{queue,media}.ts.

One-page revision sheet

A 60-second answer

"AuraSpace helps users understand and rearrange a furnished room. We preserve photos, propose furniture using pretrained instance segmentation, let users correct masks and link observations, and keep dimensions with their source. Inpainting creates optional empty-room previews. A shared geometric scene supports inside, overview and plan views, tray placement, editing and spatial warnings. Local storage keeps manual planning available offline; a private queued backend handles heavier analysis. Current limits include placeholder automatic floor corners, unverified room-scale accuracy and pending physical-phone evaluation."

Remember	Meaning in this project
OpenCV / NumPy / Pillow	Pixel operations / array math / decode and orientation
Mask R-CNN	Server labels + boxes + per-instance masks; pretrained, not custom trained
COCO-SSD	Lighter browser box detector; not native offline learned inference
Big-LaMa	Learned appearance completion; hidden content is synthesized
Depth Anything V2	Configured indoor metric depth; colors are only visualization
Observation → item → placement	Image evidence → physical identity/dimensions → layout pose
Homography	Four-point mapping of a plane; not a full 3D scan
SAT / raycasting	Rotated overlap test / screen-to-world candidate point
Docker / Compose	Service environments / coordinated local startup
Redis / BullMQ	Queue substrate / Node job scheduling and retries
PostgreSQL / SQLite	Durable server records / offline local documents
Supabase Storage / SHA-256	Private asset bytes / integrity checks, not encryption
CAS / idempotency	Reject stale writes / avoid duplicate intent for one key
Smoke test / benchmark	Shows execution / measures quality over defined independent data

Five answers to never improvise

- **Accuracy:** no verified independent detection/depth/dimension percentage is provided.
- **Dimensions:** a pixel box or category prior is not an independently measured real dimension.
- **Reconstruction:** multi-photo capture and rendered furniture do not prove an automatically reconstructed room.
- **Performance:** host smoke timings and emulator success do not establish phone FPS or camera quality.
- **Contribution:** the implemented workflow and engineering integration are the contribution; the team did not invent or train the underlying pretrained architectures.

Source trail: Refer to pages 5–41 for implementations, rationale and qualification of each statement.

Repository map for viva follow-ups

Paths are relative to the project root. Use `rg` to find the named function or field. These references make the explanation auditable without relying on line numbers that change after edits.

Location	Open it to explain
services/vision/app/preprocessing.py	EXIF-aware decode, CLAHE, grayscale, Sobel/Canny, contours, histogram and morphology
services/vision/app/inference.py	Verified Mask R-CNN load, resize, filtering, NMS, masks and coordinate restoration
services/vision/app/masks.py	Reviewed polygon/brush/box masks, expansion and cleaned cutout features
services/vision/app/inpainting.py and lama.py	Classical fallback, binary composition and Big-LaMa adapter
services/vision/app/depth.py	Metric model load, heuristic label, raw meters and millimeter PNG
services/vision/app/room_analysis.py and routes.py	Hough/vanishing helpers, unused RANSAC helper, endpoint orchestration and placeholder corners
services/vision/model-manifest.json and reports/	Pinned weights/revision/hashes and recorded execution evidence
packages/scene-domain/src/schema.ts and scene.ts	V2 data, provenance, commands, identity, fixtures, layouts and history
packages/scene-domain/src/calibration.ts	Four-corner homography validation/solve/mapping
packages/scene-domain/src/spatial.ts and legacy/geometry.ts	Concave containment, SAT, warnings, walking and circulation
packages/scene-assets/src/FurnitureModel.tsx	Procedural geometry, materials and dimensions
apps/mobile/src/features/editor/	Native scene, SVG plan, tray and item/room controls
apps/mobile/src/features/capture/	Photo capture/review, masks, calibration and processing evidence
apps/mobile/src/state/, storage/ and services/	Editor state, local SQLite/files, jobs, sync and exports
apps/api/src/routes/ and prisma/schema.prisma	Ownership-aware HTTP actions and durable data model
apps/worker/src/ and packages/backend/src/	Processor, recovery, queue, auth, configuration and Supabase adapter
src/services/vision.web.ts and vision-core.ts	Browser SSD, source mapping and lightweight pixel/fill path
src/components/scene/ and app/api/	Web cameras/rendering and server vision proxy
docker-compose.yml; infra/; Dockerfiles	Local services, build images, network, persistence and mounts

Source priority: executable source and recorded artifacts → current implementation audit → current run guide → older notes/plans. A plan describes intended work; an installed package may be unused; a comment may be stale. The project should be explained from what is actually called.

Source trail: Current repository snapshot inspected for this handbook; base commit f2a83e8. Main narrative source: docs/implementation-audit.md.

References and provenance

Primary references were checked while preparing this guide. They explain the underlying techniques; they do not certify AuraSpace's accuracy. Repository source is the authority for integration behavior. Numbered references are cited near their corresponding discussion.

Ref	Primary source
R1	OpenCV: histogram equalization and CLAHE
R2	OpenCV: Canny edge detection
R3	OpenCV: morphological transformations
R4	He et al., Mask R-CNN, arXiv:1703.06870
R5	TensorFlow.js COCO-SSD implementation and usage
R6	Suvorov et al., LaMa / Fourier inpainting, arXiv:2109.07161
R7	IOPaint LaMa adapter and export context
R8	Depth Anything V2 upstream repository and Metric Indoor Small model card
R9	Three.js Raycaster documentation
R10	React Three Fiber introduction
R11	Docker: what is a container?
R12	BullMQ: retrying failing jobs
R13	Redis persistence documentation

Image and model provenance

The visual plates use COCO val2017 image 39769, already named in the project's smoke report. It is a public model-execution fixture rather than a project-team room or an independent test set. Retain the source dataset's license/attribution records when redistributing the image in a formal publication.

The downloaded input SHA-256 matches the recorded fixture hash (`dea9e7ef...63b04e`). Processing panels call repository functions; segmentation/removal panels reuse recorded artifacts. Full checkpoint hashes and metric-depth revision are in `services/vision/model-manifest.json`; exact smoke evidence is in `services/vision/reports/`.

Rebuilding and updating this document

Editable source: `docs/viva/Viva_Guide.md`. PDF builder: `docs/viva/build_guide.py`. Run **python3 docs/viva/build_guide.py** in the prepared environment with ReportLab, Pillow, OpenCV, NumPy and the vision module's PyTorch dependency. Image plates also require the existing model-smoke artifacts. Recheck feature status and reported evidence before a later presentation.

Source trail: External links support underlying concepts. Local source trails support claims about AuraSpace; report timings remain host-specific recorded observations.